Chapter 21: Data Structures for Disjoint Sets

1) Some applications involve grouping n distinct elements into a collection of disjoint sets. Two important operations are then finding which set a given element belongs to and uniting two sets.

2) **Disjoint-set operations:** A "disjoint-set data structure" maintains a collection $\mathcal{S} = \{S_1, S_2 \ldots S_k\}$ of disjoint dynamic sets. Every set is identified by a representative, which is some member of the set.

→ In some applications, it does not matter which member is used as the representative. We only care that when we ask for the representative of a dynamic set twice without modifying the set between the requests, we get the same answer both times. In other applications, there may be a prespecified rule for choosing the representative, such as choosing the smallest member in the set.

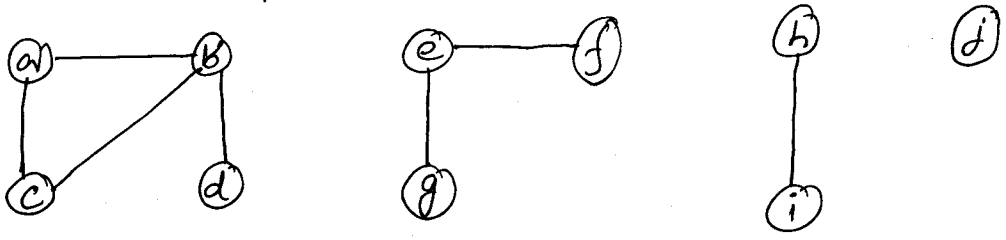
→ We wish to support the following disjoint-set operations:-

1) **MAKE_SET(x):** creates a new set whose only member (and thus representative) is x. Since the sets are disjoint, we require that x should not be there in any other set.

2) **UNION(x, y):** unites the dynamic sets that contain x and y, say S_x and S_y. The representative of the resulting set is any member of $S_x \cup S_y$, although many implementations of UNION specifically choose the representative of either S_x or S_y as the new representative. We next remove S_x and S_y from the collection so that the collection remains disjoint.

3) **FIND_SET(x):** returns a pointer to the representative of the (unique) set containing x.

3) **An application of disjoint-set data structures:** One application of disjoint-set data structures arises in determining the connected components of an undirected graph.
eg:- A graph with 4 connected components :-



The procedure CONNECTED_COMPONENTS uses the disjoint-set operations to compute the connected components of a graph :-

CONNECTED_COMPONENTS (G)

```
1  for each vertex v ∈ V[G]
2        do MAKE_SET (v)
3  for each edge (u,v) ∈ E[G]
4        do if FIND_SET (u) ≠ FIND_SET(v)
5              then UNION (u,v)
```

The procedure SAME_COMPONENT answers queries about whether 2 vertices are in the same set, i.e. same connected component :-

SAME_COMPONENT(u,v)

```
1  if FIND_SET(u) = FIND_SET(v)
2        then return TRUE
3        else return FALSE
```

→ When the edges of a graph are static i.e. not changing over time, the connected components can be computed faster by using depth-first search. However, sometimes the edges are "added" dynamically and we need to maintain the connected components as each edge is added. In this case, the implementation given here can be more efficient than running a new depth-first search for each new edge. [Just check whether the 2 endpoints fall in the same set or not.]

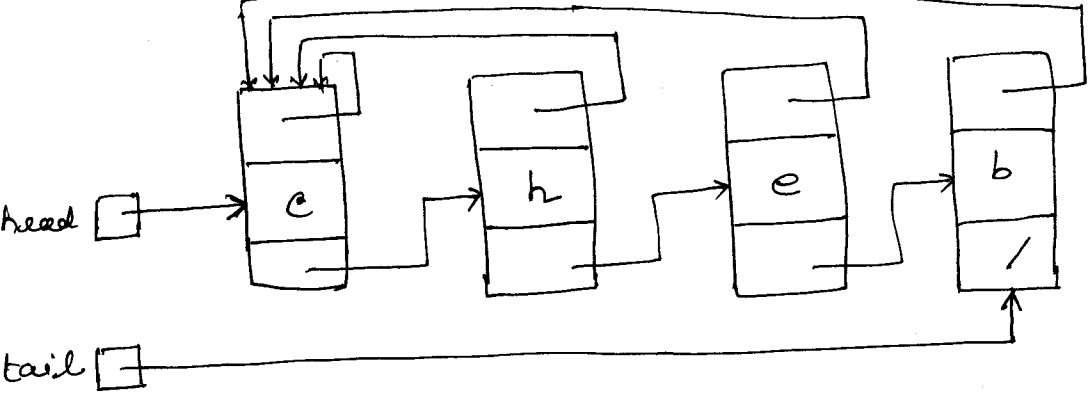
4) Linked-list representation of disjoint sets:

A simple way to implement a disjoint set data structure is to represent each set by a linked list. The first object of every linked list acts as a representative of the set the linked-list stands for. Each object of the linked-list possesses the following fields:—

(i) a set member,
(ii) a pointer to the object containing the next set member,
(iii) a pointer to the representative i.e. the first object of the linked list.

Every list object contains the following pointers:-
(i) head: points to the representative,
(ii) tail: points to the last object of the list.

Within each linked list the objects may appear in any order.



→ With linked-list representation, MAKE_SET and FIND_SET are in $O(1)$ time. To carry out MAKE_SET(x) we simply create a single node linked-list containing x. For FIND_SET(x) we simply return the pointer to the representative.

A simple implementation of union :-

We can perform UNION(x,y) by appending x's list onto the end of y's list. We use the tail pointer for y's list to quickly find out where to append x's list.

The representative of the new set is the element that originally was the representative of the set containing y. We must update the pointer to the representative for every object that originally was on x's list.

The time required by the procedure would be linear in the length of x's list.

→ It is not difficult to come up with a sequence of "m operations" on "n objects" that requires $\Theta(n^2)$ time (i.e. m operations which run in time quadratic in the no. of objects (i.e. nodes) they affect). Suppose a sequence of operations where

you execute n MAKE-SET operations followed by $n-1$ UNION operations. Then $m = 2n-1$. [A sequence of n MAKE-SET operations produces n singleton sets so that a maximum of $n-1$ UNION operations can be executed.]

Operation	No. of objects updated
MAKE_SET(x_1)	1
MAKE_SET(x_2)	1
⋮	⋮
MAKE_SET(x_n)	1
UNION(x_1, x_2)	1
UNION(x_2, x_3)	2
UNION(x_3, x_4)	3
⋮	
UNION(x_{n-1}, x_n)	$n-1$

→ Time spent performing MAKE_SET operations = $\Theta(n)$

→ Because the ith UNION operation updates i objects, the total no. of objects updated by $n-1$ UNION operations is $= \sum_{i=1}^{n-1} i = \Theta(n^2)$ = time required.

→ Since the total no. of operations is $2n-1$, each operation requires $\Theta(n)$ time. That is, the "amortized time" of an operation is $\Theta(n)$.

A weighted-union heuristic:

In the worst case, we may be appending a longer list to a shorter list and the average time required is $\Theta(n)$ per call. For example, in the case presented previously the total time for a sequence of m operations is $\Theta(n^2)$, where n is the no. of objects you start with.

A better method is to append the smaller list onto the larger. In this case each list needs to maintain the length of the list. This is called the "weighted-union heuristic".

With this heuristic a single union operation can still take $\Omega(n)$ time if both sets have $\Omega(n)$ members.

Theorem: Using the linked-list representation of disjoint sets and the weighted-union heuristic, a sequence of m MAKE_SET, UNION and FIND_SET operations, n of which are MAKE_SET operations takes $O(m + n\lg n)$ time.

Proof: For every object in a set of size n, we start by computing an upper bound on the no. of times the object's pointer back to the representative has been updated.

Consider an object x. We know that each time x's representative pointer was updated, x must have started in the smaller set. Therefore, the first time x's representative pointer was updated the resulting set must have had at least 2 members.

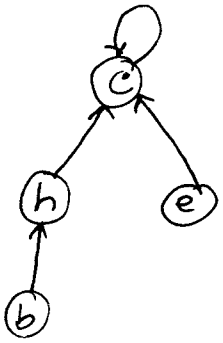
The next time x's representative pointer was updated, the resulting set must have had at least 4 members. For any $k \leq n$, after x's representative pointer has been updated $\lceil \lg k \rceil$ times, the resulting set must have at least k members. Since the largest set contains at most n members, each objects representative pointer has been updated at most $\lceil \lg n \rceil$ times over all the UNION operations. The total time used in updating the n objects is thus $O(n \lg n)$.

Each make-set and FIND-SET operation takes $O(1)$ time, and there are $O(m)$ of them. Thus, the total time for entire sequence is $O(m + n \lg n)$. (which is better than the $O(n^2)$ time ~~for th~~ without using the weighted union heuristic).

5) Disjoint-set forests :

We can represent disjoint sets by rooted trees. Each node of a tree contains one member and each tree represents one set. In a disjoint-set forest each member points only to its parent.

for eg:-



$\{b, c, e, h\}$

$\{d, f, g\}$

The root of every tree contains the representative and is its own parent.

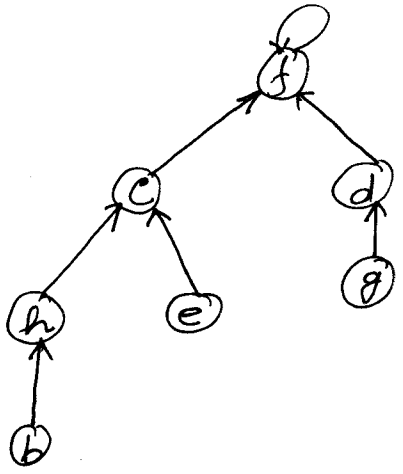
→ The straightforward algorithms that use this representation are no faster than ones that use the linked-list representation. But by including two heuristics – "union by rank" and "path compression" – we can achieve the asymptotically fastest disjoint-set data structure known.

→ Disjoint-set operations :

1) MAKE-SET operation simply creates a tree with one node.

2) FIND-SET operation traverses the parent pointers until we reach the root of the tree. The nodes visited on this path toward the root constitute the "find path".

3) UNION operation simply causes the root of one tree point to the root of the other.
For eg: UNION of previously shown 2 trees is :-

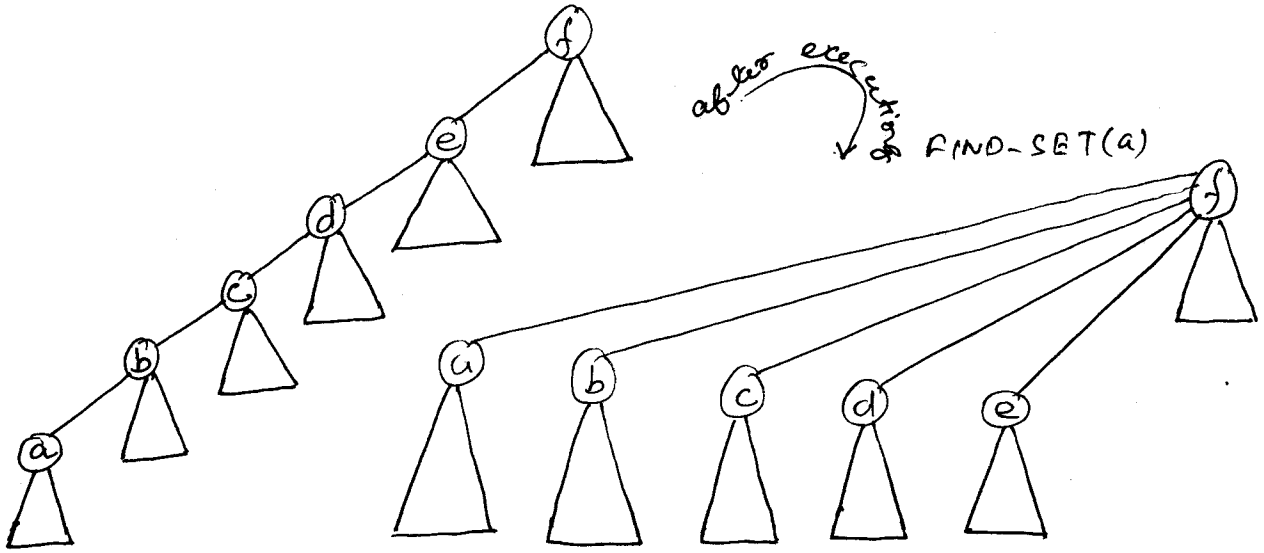


$\{b, c, e, h, d, f, g\}$

→ So far we have not improved on the linked list implementation. A sequence of $m-1$ UNION operations may create a tree that is just a linear chain of n nodes.

→ **The union by rank heuristic :-** The idea is to make the root of the tree with fewer nodes point to the root of the tree with more nodes. Rather than explicitly keeping track of the size of the subtree rooted at each node, we associate with every node a "rank" which is an upper bound on the height of the node. In union by rank, the root with smaller rank is made to point to the root with larger rank during a UNION operation.

→ **The path compression heuristic :-** The path compression heuristic is applied during FIND-SET operations to make every node on the "find path" point directly to the root. Path compression does not change any ranks.



→ With each node x, we maintain an integer value rank[x], which is an upper bound on the height of x. When a singleton set is created by MAKE-SET, the initial rank of the single node in the corresponding tree is 0. A FIND-SET operation leaves all ranks unchanged. The UNION operation may or may not affect

the ranks. We have two cases:-

(i) If the roots of the two trees have unequal rank, the root with smaller rank is made to point to the root with larger rank. The ranks themselves remain unchanged.

(ii) If the roots have equal rank, we arbitrarily choose one of the roots as the parent and increment its rank.

MAKE_SET(x)

```
1  parent[x] ← 0
2  rank[x] ← 0
```

UNION(x,Y)

```
1  LINK( FIND-SET(x), FIND-SET(y))
▷ To the link procedure, you pass the representatives
▷ of the 2 sets you want to unite
```

The FIND-SET procedure with path compression is as follows:-

LINK(x,y)

```
1  if rank[x] > rank[y]
2       then  parent[y] ← x
3       else  parent[x] ← y
4             if rank[x] = rank[y]
5                then rank[y] ← rank[y] + 1
```

FIND-SET(x)

```
1  if  x ≠ p[x]        ▷ p[x] is parent of x
2       then  p[x] ← FIND-SET(p[x])
```

3 return $p[x]$

The FIND-SET procedure is a two pass method : it makes one pass up the find path to find the root, and it makes a second ~~path~~ pass back down the find path to update each node and make its parent pointer point to the root.